ESHRAGH UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

A Comparative Analysis of Stride Scheduling, Rate Monotonic, and Earliest Deadline First Algorithms

Proportional-Share vs. Fixed-Priority vs. Dynamic-Priority CPU Scheduling

STUDENT INFORMATION

Full Name: *Zeynab Rezaii*
Student ID: *1403176010J*
Course Name: *Majid Shamshirgaran*

INSTRUCTOR & SUBMISSION

Instructor: *Majid Shamshirgaran*
Submission Date: *10/20/2026*

Table of Contents

1. Abstract
2. Introduction
3. Overview of Stride Scheduling (SS)
4. Overview of Rate Monotonic (RM)
5. Overview of Earliest Deadline First (EDF)
6. Comparative Analysis
7. Illustrative Example
8. Discussion
9. Conclusion
10. References

SECTION 1 Abstract

This paper presents a concise comparative analysis of three influential CPU scheduling algorithms: Stride Scheduling (SS), Rate Monotonic (RM), and Earliest Deadline First (EDF). Although all three algorithms address the problem of allocating processor time among competing tasks, they do so with fundamentally different objectives. Stride Scheduling is a deterministic proportional-share algorithm that allocates CPU time in proportion to ticket counts and offers no inherent deadline guarantees. Rate Monotonic is a fixed-priority real-time algorithm that assigns higher priorities to periodic tasks with shorter periods and provides a classical sufficient schedulability bound for independent periodic task sets. Earliest Deadline First is a dynamic-priority real-time algorithm that selects the ready job with the earliest absolute deadline and can achieve full utilization under the classical ideal model. This paper compares the three algorithms across objective, scheduling principle, priority model, deadline awareness, fairness, preemption, and computational overhead, and illustrates their differences through a small, clearly labeled hand-calculated example. The paper concludes that no algorithm is universally superior: SS suits weighted resource sharing, RM suits predictable fixed-priority periodic workloads, and EDF suits dynamic periodic workloads where maximum utilization is required under the classical assumptions.

Keywords: Stride Scheduling, Rate Monotonic, Earliest Deadline First, Proportional-Share Scheduling, Real-Time Scheduling, CPU Scheduling, Schedulability.

SECTION 2 Introduction

CPU scheduling is a core responsibility of every operating system. When multiple tasks are ready to run, the scheduler must decide which one to execute and for how long. The design of a scheduling policy directly affects system throughput, responsiveness, fairness, and predictability. Classical general-purpose schedulers such as First-Come, First-Served and Round Robin were designed for time-sharing systems, but they do not naturally support either precise proportional resource allocation or guaranteed real-time behavior.

Two distinct families of scheduling algorithms emerged to address these limitations. *Proportional-share scheduling* allows clients to specify relative entitlements, and the scheduler attempts to allocate resources in proportion to those entitlements. *Real-time scheduling* addresses systems in which tasks have timing constraints and must complete by specified deadlines. Stride Scheduling belongs to the first family; Rate Monotonic and Earliest Deadline First belong to the second. The purpose of this paper is to compare these three algorithms in a structured and technically accurate way, emphasizing that they solve different scheduling problems rather than competing directly for the same objective.

The remainder of the paper is organized as follows. Sections 3–5 describe the three algorithms. Section 6 provides a comparative analysis, Section 7 presents an illustrative example, Section 8 discusses the practical implications, Section 9 concludes, and Section 10 lists the references.

SECTION 3 Overview of Stride Scheduling (SS)

Purpose and Basic Principle

Stride Scheduling, introduced by Waldspurger and Weihl as a deterministic companion to lottery scheduling, is a proportional-share resource allocation algorithm. Its purpose is to distribute a shared resource—most commonly CPU time—among competing clients in proportion to their ticket allocations. Each runnable process maintains a *pass* value that represents its accumulated service in a virtual time domain. At each scheduling decision, the process with the smallest pass value is selected to run for one quantum, after which its pass value is advanced by its stride.

Ticket Allocation, Stride, and Pass Value

Tickets express relative entitlements: if process A holds twice as many tickets as process B, then A is entitled to roughly twice as much CPU time. The stride is derived from the ticket count:

EQUATION 1 — STRIDE

$$\text{stride}_i = K \,/\, \text{tickets}_i$$

where K is a configurable large constant. After a process is selected and runs for one quantum, its pass value is updated:

EQUATION 2 — PASS UPDATE

$$\text{pass}_i \leftarrow \text{pass}_i + \text{stride}_i$$

Under the basic algorithm, the runnable process with the smallest pass value is selected next. This deterministic rule ensures that the pass values of continuously runnable processes remain approximately balanced, and that the long-term fraction of CPU time received by a process converges to its ticket share.

Advantages, Limitations, and Applications

The principal advantages of Stride Scheduling are precise proportional sharing, deterministic decision-making, a simple mathematical foundation, and flexible weighting through tickets. Its limitations include dependence on correct ticket allocation, sensitivity to initialization policies, potential numerical drift in fixed-point implementations, and challenges in multicore environments. Crucially, Stride Scheduling does not inherently guarantee that real-time deadlines will be met; it is a proportional-share algorithm, not a real-time scheduler. It is therefore appropriate for weighted resource allocation, operating-system research, educational simulations, and controlled CPU-share experiments, not for hard real-time workloads.

SECTION 4 Overview of Rate Monotonic (RM)

Purpose and Fixed-Priority Principle

Rate Monotonic is a classical fixed-priority real-time scheduling algorithm for periodic tasks. Each task has a fixed period T_i and a worst-case execution time C_i. RM assigns fixed priorities according to task periods: tasks with shorter periods receive higher priorities. The priority of a task does not change over time. RM is typically used with preemptive scheduling, in which a newly released high-priority task can interrupt a running lower-priority task.

Schedulability Analysis

Under the standard Liu–Layland model—independent periodic tasks, preemptive fixed-priority scheduling, deadlines equal to periods, and negligible scheduling overhead—RM is schedulable if the total CPU utilization satisfies the sufficient condition:

EQUATION 3 — LIU–LAYLAND UTILIZATION BOUND

$$U \le n \cdot (2^{1/n} - 1)$$

where $U = \Sigma\, C_i \,/\, T_i$ is the total utilization and n is the number of tasks. As $n \to \infty$, the bound converges to $\ln 2 \approx 0.693$. It is essential to note that this is a *sufficient* condition, not a necessary one: some task sets with utilization above the bound are still schedulable under RM, and exact schedulability can be determined using response-time analysis.

Advantages, Limitations, and Applications

RM is simple, analyzable, and well suited to systems with predictable periodic workloads, such as embedded control systems. Its fixed-priority structure simplifies implementation and reasoning. Its main limitations are the uti-

lization bound below 100% for large task sets and the risk of starvation for low-priority tasks under overload. RM is not designed for proportional resource sharing or for aperiodic workloads.

SECTION 5 Overview of Earliest Deadline First (EDF)

Purpose and Dynamic-Priority Principle

Earliest Deadline First is a dynamic-priority real-time scheduling algorithm. At every scheduling point, EDF selects the ready job with the earliest absolute deadline. Priorities are not fixed; they change dynamically as jobs are released and as time advances. EDF is typically used with preemptive scheduling.

Schedulability Conditions

For the classical uniprocessor model with independent periodic tasks, preemptive scheduling, deadlines equal to periods, and negligible overhead, the total utilization is:

EQUATION 4 — TOTAL UTILIZATION

$$U = \Sigma_i \, C_i / T_i$$

Under these ideal assumptions, EDF can feasibly schedule any task set with $U \leq 1$; that is, EDF achieves 100% utilization and is optimal among uniprocessor preemptive schedulers for this model. In practical systems, scheduling overhead, blocking, non-preemptive sections, and different deadline models can reduce the achievable utilization.

Advantages, Limitations, and Applications

EDF's main advantage is its ability to achieve full processor utilization under the classical model and its dynamic adaptivity to changing deadlines. Its main limitations include increased complexity, less predictable behavior under overload (the "domino effect" can cause multiple tasks to miss deadlines), and reduced robustness compared to fixed-priority schemes. EDF is appropriate for real-time systems with dynamic periodic or aperiodic workloads, particularly those with utilization requirements above the RM bound.

SECTION 6 Comparative Analysis

The three algorithms address different scheduling problems. Stride Scheduling is a proportional-share algorithm, whereas RM and EDF are real-time algorithms. The comparison below therefore focuses on clarifying their distinct objectives and assumptions rather than treating them as interchangeable alternatives.

Table 1. Comparison of Stride Scheduling, Rate Monotonic, and Earliest Deadline First

Criterion	Stride Scheduling (SS)	Rate Monotonic (RM)	Earliest Deadline First (EDF)
Primary objective	Proportional CPU-share allocation	Meet deadlines for periodic tasks (fixed-priority)	Meet deadlines (dynamic-priority, optimal)
Scheduling principle	Select minimum pass; pass += stride	Highest priority = shortest period	Earliest absolute deadline first
Priority model	Ticket-based weights (not priorities)	Fixed priority determined by period	Dynamic priority determined by deadline
Ticket allocation	Central: tickets express entitlements	Not applicable	Not applicable
Deadline awareness	None	Implicit via periods	Explicit; deadlines drive selection
Real-time guarantees	None	Yes (under Liu–Layland assumptions)	Yes (up to U = 1 under classical model)
Fairness	Proportional fairness	Priority-based; starvation possible for low-priority tasks	Deadline-based; low-deadline tasks favored
Preemption	Yes, at quantum boundaries	Yes (preemptive variant)	Yes
Computational overhead	O(log n) with heap/BST; O(n) linear	O(1) selection with fixed priorities	O(log n) with priority queue
Main advantages	Precise proportional shares; deterministic	Simple, predictable, analyzable	Optimal utilization under ideal model
Main limitations	No deadline support; multicore challenges	Utilization bound below 100%; priority inversion	Overload domino effect; less predictable
Typical use cases	Weighted resource sharing, research	Embedded control, periodic fixed-priority systems	Real-time systems with dynamic deadlines

> **Key Distinction:** Stride Scheduling addresses *proportional sharing*; RM and EDF address *deadline satisfaction*. Treating tickets as equivalent to periods or deadlines is conceptually incorrect and would lead to misleading comparisons.

SECTION 7 Illustrative Example

Because SS and real-time algorithms have different assumptions, the example below uses a proportional-share workload for SS and a periodic real-time workload for RM and EDF. The purpose is to illustrate each algorithm's behavior, not to compare them under identical assumptions.

Stride Scheduling Example

Consider three continuously runnable processes with tickets $t_1 = 50$, $t_2 = 30$, and $t_3 = 20$. Let $K = 3000$. The strides are $s_1 = 60$, $s_2 = 100$, and $s_3 = 150$. The theoretical shares are 50%, 30%, and 20%, respectively. With all pass values initialized to 0, the first several selections are P1, P2, P3, P1, P1, P2, P1, P3, ... which approximately follows the ticket proportions. Over a long interval, the observed share of each process converges to its ticket share.

Rate Monotonic and EDF Example

Consider three periodic tasks under the classical model (independent, preemptive, deadlines = periods, negligible overhead):

- Task A: $C_A = 1$, $T_A = 4$, $U_A = 0.25$
- Task B: $C_B = 2$, $T_B = 5$, $U_B = 0.40$
- Task C: $C_C = 2$, $T_C = 10$, $U_C = 0.20$

The total utilization is $U = 0.85$. The Liu–Layland bound for $n = 3$ is $3(2^{1/3} - 1) \approx 0.780$. Since $U = 0.85 > 0.780$, the sufficient RM condition is not met; however, this does not prove infeasibility. Response-time analysis or simulation would be needed to decide schedulability for this task set under RM. Under EDF, since $U = 0.85 \leq 1$ and the classical assumptions hold, the task set is schedulable.

Table 2. Illustrative Comparison of Algorithm Objectives (Hand-Calculated)

Algorithm	Input Model	Selection Rule	Guarantee
SS	Tickets per process	Minimum pass value	Long-term proportional share
RM	(C_i, T_i) per task	Highest fixed priority (shortest period)	Feasibility if $U \le n(2^{1/n} - 1)$
EDF	(C_i, T_i, D_i) per task	Earliest absolute deadline	Feasibility if $U \le 1$ (classical model)

SECTION 8 Discussion

The comparison makes clear that the three algorithms are not interchangeable. Stride Scheduling is the appropriate choice when the goal is to allocate CPU time in proportion to client-defined weights, and when no deadlines need to be guaranteed. Its determinism and mathematical clarity make it attractive for research, education, and weighted resource allocation. However, it offers no real-time guarantees and its multicore behavior requires careful design.

Rate Monotonic is appropriate for periodic real-time systems with predictable workloads and fixed priorities. It is simple, analyzable, and widely supported in embedded and industrial systems. Its main drawback is the conservative utilization bound and the possibility of starvation for low-priority tasks under overload. When utilization exceeds the Liu–Layland bound, RM may still be feasible, and exact schedulability should be determined with response-time analysis.

EDF is preferable when the task set requires utilization close to 100% under the classical model, or when deadlines are not necessarily equal to periods and must be tracked explicitly. Its dynamic-priority nature allows it to adapt to changing workloads, but it is less robust under overload than fixed-priority schemes, where the lowest-priority tasks fail first. In practice, the choice between RM and EDF depends on the system's tolerance for overload behavior and on the specific task model.

The significance of workload assumptions cannot be overstated. The RM utilization bound and the EDF full-utilization result both rely on the classical Liu–Layland model. Overhead, blocking, non-preemptive sections, and non-periodic workloads can invalidate these guarantees. Similarly, Stride Scheduling's proportional-share guarantees assume stable tickets and a continuously runnable process set. No algorithm should be considered universally best; the appropriate choice depends on the scheduling objective, the workload model, and the system's tolerance for particular failure modes.

SECTION 9 Conclusion

This paper has compared Stride Scheduling, Rate Monotonic, and Earliest Deadline First. Stride Scheduling is a deterministic proportional-share algorithm that allocates CPU time in proportion to ticket counts and provides no deadline guarantees. Rate Monotonic is a fixed-priority real-time algorithm that assigns higher priorities to tasks with shorter periods and provides a classical sufficient schedulability bound. Earliest Deadline First is a dynamic-priority real-time algorithm that selects the job with the earliest absolute deadline and achieves full utilization under the classical ideal model.

The main conclusion is that the three algorithms solve different problems and should be selected according to the system's primary objective. Stride Scheduling is appropriate for weighted resource sharing, RM is appropriate for periodic fixed-priority real-time workloads, and EDF is appropriate for dynamic real-time workloads that require high utilization under the classical assumptions. No single algorithm dominates the others, and the choice among them must be guided by an explicit analysis of the workload model, the required guarantees, and the system's tolerance for overhead and overload behavior.

SECTION 10 References

1. C. A. Waldspurger and W. E. Weihl, "Stride scheduling: Deterministic proportional-share resource management," MIT Laboratory for Computer Science, Cambridge, MA, USA, Technical Memorandum MIT/LCS/TM-528, Jun. 1995.

2. C. A. Waldspurger and W. E. Weihl, "Lottery scheduling: Flexible proportional-share resource management," in *Proc. First Symp. Operating Systems Design and Implementation (OSDI)*, Monterey, CA, USA, Nov. 1994, pp. 1–11.

3. C. A. Waldspurger, "Lottery and stride scheduling: Flexible proportional-share resource management," Ph.D. dissertation, Dept. Elect. Eng. Comput. Sci., Massachusetts Institute of Technology, Cambridge, MA, USA, Sep. 1995.

4. C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, Jan. 1973.

5. G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 3rd ed. New York, NY, USA: Springer, 2011.

6. A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

7. R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*. Madison, WI, USA: Arpaci-Dusseau Books, 2018.

8. J. W. S. Liu, *Real-Time Systems*. Upper Saddle River, NJ, USA: Prentice Hall, 2000.

— End of Paper —